

Alberta Buck – Identity Requirements and Cryptographic Receipts

Perry Kundert

2026-03-14

Every operation in the Alberta Buck ecosystem has identity requirements. Some operations require fully public identity (oracle claims, governance votes). Others require only that the participant is a verified identity, without disclosing who they are (BUCK transfers). DeFi integrations (Uniswap pools, lending protocols) introduce contract addresses as counterparties, which also need identities associated with their deployers.

Identity credentials are ElGamal ciphertexts encrypted under the participant’s public key, signed by the issuing authority’s private key, and published to the issuer’s Holochain source chain – immutable, auditable, and verifiable against the issuer’s well-known public key. All participants use ElGamal over the same elliptic curve group (alt_bn128), enabling Chaum-Pedersen proofs of correct re-encryption: when Alice must share her identity with counterparty Bob, she re-encrypts her credential under Bob’s public key and produces a proof – verifiable by anyone – that the re-encrypted ciphertext contains the same identity as her issuer-signed credential, without revealing the identity itself. On-chain verification costs ~27,000 gas (3-4 elliptic curve scalar multiplications via Ethereum precompiles), compared to ~200,000+ gas for SNARK-based alternatives.

This document tabulates the identity requirements for every operation, details the ElGamal Chaum-Pedersen cryptographic architecture, and demonstrates through attack scenarios that the scheme defeats identity forgery, substitution, and collusion attacks while maintaining very low on-chain costs.

Contents

1	Identity with Anonymity	3
1.1	Revealing Identity	3
1.2	Accounts Belong to Someone	3
1.3	Counterparties Exchange Identities	4
1.4	ElGamal Over a Mandated Curve Group	5
1.5	Chaum-Pedersen Proof of Re-Encryption Correctness	6
2	Attacks and Defenses	7
2.1	Attack 1: Alice Substitutes a False Identity for Bob	8
2.2	Attack 2: Alice Forges an Issuer Credential	9

2.3	Attack 3: Alice and Complicit Bob Launder With False Identities	9
2.4	Attack 4: Alice Repudiates a Transaction She Made	10
2.5	Attack 5: Criminal Bob Denies Receiving Alice’s Identity	11
2.6	Attack 6: Eve Links Alice’s Identities Across Counterparties	12
2.7	Attack 7: Alice Uses a Revoked or Expired Credential	13
2.8	Attack 8: Unwitting Bob Receives a Transaction From Criminal Alice . .	13
2.9	Summary: Why Chaum-Pedersen Defeats All Substitution Attacks	14
3	Identity Visibility Modes	15
4	Operations Table	15
4.1	Core BUCK (ERC-20) Operations	15
4.2	BuckCredit (ERC-721) Operations	15
4.3	BuckKController (PID Stabilization)	16
4.4	Oracle Operations	16
4.5	DeFi Integration (Uniswap, Lending, etc.)	16
4.6	Identity Management	17
5	Contract Identity: How DeFi Pools Become BUCK Counterparties	17
6	The Receipt Problem: What Can Be Public, What Cannot	18
6.1	The Fundamental Challenge: Proving Re-Encryption Correctness	18
6.2	Homomorphic Equality Check Without Decryption	20
6.3	The <code>approve</code> Function as Identity Handshake	21
6.4	Receipt Construction: What the Contract Emits	22
6.5	The Complete Flow	23
6.6	Summary: What Each Layer Sees	23
7	Cryptographic Receipts: 3-Party ECDH	24
7.1	The Three Keys	24
7.2	Encryption Scheme	24
7.3	Who Can Decrypt	25
8	Play-by-Play: BUCK/USDT Swap on Uniswap	26
8.1	Prerequisites (One-Time Setup)	26
8.2	The Swap Transaction	26
9	Edge Cases and Design Considerations	30
9.1	Flash Loans and Atomic Transactions	30
9.2	Multi-Hop Swaps	30
9.3	Contract-to-Contract Transfers	30
9.4	Wrapping and Bridging	31
9.5	Gas Costs	31
9.6	Privacy of Pool Deployer Identity	31
10	Summary: Identity Requirement by Risk Level	31

1 Identity with Anonymity

The expectation to be able to transact in complete freedom (no one else can prevent it) and anonymity (no one else can identify the parties) is paramount. This is consistent with the expectation that one is innocent until proven guilty. Asking permission from anyone before we can buy, sell or trade is unacceptable to any free people. Anything less turns a free citizen into a serf. If the government can lock your account and freeze you on a park bench at will, how free are you?

However, the expectation that I can trust that my counterparty's identity is *at least* as reliable as the group(s) it is part of is also critical. Allowing arbitrary hostile groups to ravage a community under the cover of darkness has been used throughout history to subjugate free people. If the government can issue identity documents to whomever they wish, and you are forced to trust them, how free are you?

The Alberta Buck enforces these principles.

1.1 Revealing Identity

If I am suspected of a crime, an Alberta court can compel me to decrypt transactions they suspect are related to that crime, and reveal the identity of counterparties I've transacted with. I may resist doing so, and face the consequences: the organization that the Alberta government tasks with overseeing the authenticity of my Alberta identity cannot simply authorize decryption of my transaction counterparties! They can only vouch for my (decrypted) identity, just as I can vouch for my daughter's.

Only the participants in a transaction can decrypt the re-encrypted identity blobs exchanged during that transaction. Each re-encrypted identity is an ElGamal ciphertext under the specific counterparty's public key – only that counterparty's private key can recover the plaintext. The identity issuer cannot decrypt it (they encrypted under Alice's public key, not their own). The government cannot decrypt it. No registry operator or Holochain node can decrypt it. A court must compel a *specific participant* to decrypt a *specific transaction's* identity data. The attack cost is $O(N)$: each individual's credential must be attacked separately, rather than compromising a single custodian.

1.2 Accounts Belong to Someone

I may be a citizen of Alberta and have a minor child. My child's identity is vouched by me, and I am vouched by Alberta. I have several accounts associated with encrypted copies of my identity (income, savings, etc) and several unencrypted (walking around money). I can transfer money from my savings to my walking around account (anonymously, via my BUCK account at ATB).

My encrypted savings account Identity has been authorized by ATB to send funds. My walking around money account has a plaintext Identity, so Starbucks' BUCK account doesn't need to pre-authorize incoming transactions – it accepts BUCK transfers from any BUCK account with a public identity.

This just cuts out the middleman, and gives every citizen what government and institutions already have: knowledge of the participants in transactions with public accounts.

If I prefer, I can set up a Starbucks account, and ask them to authorize me to send money to them. Again, allowing with BUCKs what is already allowed with a pre-loaded Starbucks card – private transactions with a public account. Starbucks and Visa both know who paid to fund the Starbucks card, and who funded their Starbucks account with that card. Now, citizens can set up similar arrangements and retain that knowledge amongst themselves.

If a court compels one of the participants (for example, as the result of an arrest in a sting operation against some members of a group performing some illicit activity), then that persons transactions with other members of the group can be discovered. Otherwise: *financial activity remains confidential, at the sole choice the participants.*

An expectation that any government or bank functionary (or any random attacker, due to government or bank incompetence) can know the details of a citizens financial activities is unacceptable. The attack cost is $O(N)$ vs. $O(1)$; every individual’s credential must be attacked, instead of just being able to attack or compel one disgruntled government or bank employee.

1.3 Counterparties Exchange Identities

Each transaction is between parties identified to each other by provably untampered copies of their identity, encrypted by each party specifically for that counterparty.

The identity credential lifecycle:

1. **Issuance:** A trusted issuer (the Alberta government, ATB Financial, etc.) verifies Alice’s identity through a KYC ceremony. The issuer encrypts Alice’s identity data I under Alice’s public key using ElGamal over alt_bn128 :

$$E_alice = (r * G, r * pk_alice + I * G)$$

The issuer signs this ciphertext with their well-known private key:

$$\sigma = \text{Sign}(sk_issuer, E_alice)$$

The signed credential (E_alice, σ) is published to the issuer’s Holochain source chain, where it is immutable, auditable by anyone, and verifiable against the issuer’s well-known public key. A reference (hash of E_alice) is registered in the on-chain IdentityRegistry alongside Alice’s Ethereum address.

2. **Re-encryption:** When Alice must identify herself to counterparty Bob (e.g., during an `approve` call on the BUCK contract), she decrypts her credential locally (she knows sk_alice) and re-encrypts the identity under Bob’s public key with fresh randomness:

$$E_bob = (r' * G, r' * pk_bob + I * G)$$

She produces a Chaum-Pedersen proof π that E_bob encrypts the same point $I * G$ as her issuer-signed E_alice – without revealing I .

3. **Verification:** Anyone (including the BUCK smart contract) can verify:

- **sigma** is a valid signature from the issuer on **E_alice** (the credential is genuine, not self-issued)
- **pi** proves **E_bob** encrypts the same plaintext as **E_alice** (Alice did not substitute false data)

Verification requires only 3-4 elliptic curve scalar multiplications (~27,000 gas on Ethereum via the `alt_bn128` precompiles). No SNARK circuits, no trusted setup, no pairing operations.

4. **Decryption:** Only Bob can decrypt **E_bob** (he has **sk_bob**). Only Alice can decrypt **E_alice** (she has **sk_alice**). The issuer cannot decrypt either (they encrypted under Alice’s public key, not their own). No third party can decrypt any re-encrypted copy.

There is no need for Proxy Re-Encryption (PRE), because Alice is online when she calls **approve** and can perform the re-encryption directly. The Chaum-Pedersen proof is the trust anchor: it is a mathematical guarantee – unforgeable under the discrete logarithm assumption – that Alice re-encrypted her genuine, issuer-signed identity and not a substitute.

1.4 ElGamal Over a Mandated Curve Group

Alberta Buck mandates that all identity credentials, all participant key pairs, and all re-encryption operations use ElGamal over the `alt_bn128` elliptic curve – the curve supported by Ethereum’s `ecAdd` / `ecMul` precompiles at addresses `0x06` and `0x07`. This mandate is a design strength, not a limitation:

- **Uniform proof verification:** every Chaum-Pedersen proof uses the same group, so a single on-chain verifier handles all identity proofs.
- **Precompile efficiency:** `alt_bn128` scalar multiplication costs 6,000 gas via the EVM precompile, vs. ~100,000+ gas for in-contract EC math.
- **No cross-group compatibility issues:** since all participants use the same curve, re-encryption between any two participants is always algebraically compatible.

Participant key pairs for identity purposes are distinct from their Ethereum signing keys (which use `secp256k1`). The identity key pair is generated during the KYC ceremony and the public key is registered in the `IdentityRegistry` alongside the participant’s Ethereum address.

When Alice re-encrypts for *N* different counterparties, each re-encryption uses independent randomness. The resulting ciphertexts are IND-CPA secure: computationally indistinguishable from random curve points. An observer who collects all *N* ciphertexts cannot determine that they encrypt the same plaintext (without access to a counterparty’s private key).

1.5 Chaum-Pedersen Proof of Re-Encryption Correctness

The Chaum-Pedersen protocol is a sigma protocol (Schnorr family) that proves two ElGamal ciphertexts encrypt the same message point, without revealing the message. Made non-interactive via the Fiat-Shamir heuristic, the proof is compact (~128 bytes: two scalars and two curve points) and verification requires only 3-4 multi-scalar multiplications.

1.5.1 Setup

Issuer-signed credential (public, on issuer's Holochain source chain):

```
E_alice = (R_a, C_a) = (r_a * G, r_a * pk_alice + M)
where M = I * G (identity encoded as a curve point)
sigma = Sign(sk_issuer, E_alice)
```

Alice's re-encryption for Bob:

```
E_bob = (R_b, C_b) = (r_b * G, r_b * pk_bob + M)
using the SAME message point M
```

1.5.2 What Alice Proves (Non-Interactively)

"I know (sk_alice, r_b) such that:

1. $pk_alice = sk_alice * G$ -- I own this public key
2. $R_b = r_b * G$ -- I know E_bob's randomness
3. $C_a - sk_alice * R_a = C_b - r_b * pk_bob$ -- same message point M"

Statement 3 is the core. The left side decrypts E_alice (only Alice can compute this, since she knows sk_alice), yielding the message point M. The right side extracts M from E_bob (only Bob could alternatively compute this with sk_bob). If Alice encrypted different data I_fake, then $I * G \neq I_fake * G$ and no valid proof exists – it is not merely hard to compute, it is mathematically impossible under the discrete logarithm assumption.

1.5.3 Verification (What the Contract Computes)

Given public inputs:

```
E_alice = (R_a, C_a)    -- from IdentityRegistry (issuer-signed, read
                        -- from storage -- NOT supplied by Alice)
E_bob    = (R_b, C_b)    -- from Alice's approve() calldata
pk_alice                -- from IdentityRegistry
pk_bob                  -- from IdentityRegistry
proof    = (e, s1, s2)   -- Fiat-Shamir challenge and responses
```

Verify:

```
Recompute challenge e from commitments
Check consistency of s1, s2 against e, the public keys, and ciphertexts
```

3-4 ecMul + 3-4 ecAdd operations

Critical implementation requirement: `E_alice` is **read from on-chain storage** (the IdentityRegistry), not supplied by Alice as calldata. This anchors the proof to the issuer-signed credential. If Alice could supply her own `E_alice`, she could substitute a fake credential and produce a valid proof against it.

1.5.4 Gas Cost

Operation	Count	Gas each	Total
ecMul (precompile)	4	6,000	24,000
ecAdd (precompile)	4	150	600
keccak256 (Fiat-Shamir)	1	36	36
Calldata (128 bytes)			2,048
Storage read (<code>E_alice</code>)	1	2,100	2,100
Total			~29,000 gas

Compare: Groth16 SNARK verification requires a pairing check (113,000 gas for the `~ecPairing` precompile alone) plus additional EC operations, totaling ~200,000-300,000 gas. The Chaum-Pedersen approach is **7-10x cheaper**.

The `approve` call (which includes the proof verification) costs 29,000 gas for the identity proof plus the standard ERC-20 `approve` cost (~46,000 gas), totaling ~75,000 gas. This is paid once per counterparty pair. Subsequent `~transfer` and `transferFrom` calls emit only minimal receipt events (addresses + opaque hashes, ~2,000 gas) with no additional proof work.

2 Attacks and Defenses

The following scenarios demonstrate that the ElGamal Chaum-Pedersen proof scheme defeats identity fraud, even when participants are actively criminal. In each scenario we show what the attacker attempts, why it fails mathematically, and what the on-chain verifier sees.

The trust chain in every scenario is:

Issuer signs <code>E_alice</code>	--> credential is genuine
<code>E_alice</code> stored in registry	--> contract reads the genuine credential
Chaum-Pedersen proof verified	--> <code>E_bob</code> encrypts same identity as <code>E_alice</code>
Only Bob can decrypt <code>E_bob</code>	--> identity revealed only to counterparty

Breaking any link requires breaking either ECDSA signature unforgeability or the discrete logarithm assumption on `alt_bn128` – both are standard hardness assumptions underpinning all of Ethereum’s security.

2.1 Attack 1: Alice Substitutes a False Identity for Bob

2.1.1 Scenario

Alice is a sanctioned individual. She completed KYC under her real identity and received a legitimate issuer-signed credential `E_alice` containing her true identity `I`. She now wants to transact with Bob but wants Bob (and any future court) to believe she is someone else – say, her clean associate Carol.

Alice constructs a fake re-encryption using Carol's identity `I_fake`:

$$E_{\text{bob_fake}} = (r' * G, \quad r' * pk_{\text{bob}} + I_{\text{fake}} * G)$$

She submits `E_bob_fake` to `approve()` and attempts to produce a Chaum-Pedersen proof tying it to her registry credential.

2.1.2 Why It Fails

The Chaum-Pedersen verification checks:

$$C_a - sk_{\text{alice}} * R_a == C_b - r' * pk_{\text{bob}}$$

Expanding the left side (from the issuer-signed `E_alice` in the registry):

$$\begin{aligned} (r_a * pk_{\text{alice}} + I * G) - sk_{\text{alice}} * (r_a * G) \\ = r_a * sk_{\text{alice}} * G + I * G - sk_{\text{alice}} * r_a * G \\ = I * G \end{aligned}$$

Expanding the right side (from Alice's fake `E_bob`):

$$\begin{aligned} (r' * pk_{\text{bob}} + I_{\text{fake}} * G) - r' * pk_{\text{bob}} \\ = I_{\text{fake}} * G \end{aligned}$$

The proof requires $I * G == I_{\text{fake}} * G$, which requires $I == I_{\text{fake}}$. Since $I != I_{\text{fake}}$ (Alice is not Carol), these are distinct curve points. Under the discrete logarithm assumption, Alice **cannot produce valid proof scalars** (`s1`, `s2`) that satisfy the verification equations. The proof does not exist – not "hard to find," but mathematically non-existent.

2.1.3 What the Contract Sees

`approve()` reverts: "Re-encryption proof invalid." The transaction fails on-chain. Alice cannot complete the identity handshake with false data. The revert is indistinguishable from a malformed transaction – no information about Alice's intent leaks.

2.1.4 Cost to Alice

She wasted gas on a failed transaction. Her address is still associated with her real issuer-signed credential in the registry. Nothing has changed.

2.2 Attack 2: Alice Forges an Issuer Credential

2.2.1 Scenario

Alice never completed KYC, or completed KYC but wants a credential with different identity data. She generates her own ElGamal key pair, encrypts a fabricated identity `I_fake` under her public key, and signs it herself:

```
E_alice_fake = (r * G, r * pk_alice + I_fake * G)
sigma_fake   = Sign(sk_alice, E_alice_fake)
```

She attempts to register this credential in the IdentityRegistry and publish it to a Holochain source chain.

2.2.2 Why It Fails

The IdentityRegistry accepts credentials only from **trusted issuers** – public keys registered in the TrustedIssuersRegistry (part of the ERC-3643 framework). Registration verifies:

```
SigVerify(pk_trusted_issuer, E_alice, sigma) == true
```

Alice's `sigma_fake` was signed with `sk_alice`, not with any trusted issuer's key. The signature check fails. Her credential is rejected at registration, long before any `approve` call.

Even if Alice publishes `(E_alice_fake, sigma_fake)` to a Holochain source chain, the on-chain registry will not accept it. Holochain provides immutability and auditability of the *issuer's* source chain; it does not confer trust on self-issued credentials. The trust anchor is the issuer's signature, verified on-chain.

2.2.3 What the Contract Sees

`registerIdentity()` reverts: "Invalid issuer signature." Alice is not registered; `isVerified(alice) == false`. She cannot participate in any BUCK operation requiring identity.

2.3 Attack 3: Alice and Complicit Bob Launder With False Identities

2.3.1 Scenario

Alice and Bob are both criminals. Both have legitimate issuer-signed credentials (they passed KYC under their real identities). They want to transact with each other, but want the encrypted identity records to show *different* people – so that if a court later compels Bob to decrypt Alice's identity blob, it points to an innocent third party.

They agree: Alice will encrypt a false identity `I_fake` for Bob and submit it. Bob will claim (if asked) that the decrypted identity is genuine.

2.3.2 Why It Fails

Alice's `approve()` call must pass Chaum-Pedersen verification. The contract reads `E_alice` from the `IdentityRegistry` – not from Alice's `calldata`.

```
// Inside approve():
(E_alice, sigma) = identityRegistry.getCredential(msg.sender);
require(chaumPedersenVerify(E_alice, E_bob_submitted, proof));
```

Alice cannot substitute a different `E_alice` because the contract reads it from storage. She cannot produce a valid Chaum-Pedersen proof linking her fake `E_bob` to the real `E_alice` because the message points differ (same math as Attack 1).

Bob's willingness to accept false data is irrelevant. The proof was verified against the issuer-signed credential *before* `E_bob` was stored. If `approve()` succeeded, `E_bob` necessarily contains Alice's real identity. When a court later compels Bob to decrypt, he produces Alice's true identity – regardless of any prior agreement between them.

Bob cannot produce a *different* plaintext from the same `E_bob` (ElGamal decryption is deterministic given the private key). Bob cannot claim the `E_bob` is unrelated to Alice (the on-chain `IdentityExchange` event records that Alice submitted it during `approve`, and the proof verification succeeded).

2.3.3 What the Contract Sees

If Alice submits false data: `approve()` reverts ("Re-encryption proof invalid") – same as Attack 1.

If Alice submits real data (because she has no choice): `approve()` succeeds, and `E_bob` provably contains her real identity. The laundering scheme fails silently – Alice and Bob transacted, but the identity trail is genuine and court-recoverable.

2.4 Attack 4: Alice Repudiates a Transaction She Made

2.4.1 Scenario

Alice transacted legitimately with Bob (a law-abiding merchant). Later, Alice is investigated. She claims she never transacted with Bob, or that the identity Bob received was fabricated by Bob. Alice hopes that because the re-encrypted blob is opaque to everyone except Bob, a court cannot verify its authenticity without trusting Bob's word.

2.4.2 Why It Fails

The on-chain record contains everything needed to verify, without trusting Bob:

1. The `IdentityExchange(alice, bob, E_bob)` event is on-chain, emitted during Alice's `approve()` call. It was sent from Alice's address (verified by `msg.sender`).

2. The `approve()` transaction succeeded on-chain, which means the Chaum-Pedersen proof verified against Alice's issuer-signed `E_alice` from the registry. This is an immutable execution fact.
3. Therefore `E_bob` provably encrypts the same identity as `E_alice`. This follows from the soundness of the Chaum-Pedersen protocol – it is a mathematical fact, not testimony.
4. When the court compels Bob to decrypt `E_bob`, the resulting plaintext `I` is provably Alice's issuer-certified identity. The chain of trust:

Issuer signed <code>E_alice</code>	(verified: <code>sigma</code>)
<code>E_alice</code> stored in <code>IdentityRegistry</code>	(on-chain, immutable)
Alice called <code>approve()</code> with <code>E_bob</code> + proof	(on-chain tx, <code>msg.sender = alice</code>)
Chaum-Pedersen proof verified on-chain	(execution succeeded)
$\Rightarrow$ <code>E_bob</code> encrypts Alice's real identity	(mathematical fact)
Bob decrypts <code>E_bob</code> $\rightarrow$ <code>I</code>	(compelled, deterministic)
<code>I</code> is Alice's issuer-certified identity	(proven)

2.4.3 What the Court Sees

An unbroken, cryptographically verified chain from issuer to decrypted identity. Alice's repudiation is contradicted by on-chain proof verification records that she cannot alter or dispute.

2.5 Attack 5: Criminal Bob Denies Receiving Alice's Identity

2.5.1 Scenario

Bob is a criminal merchant. Alice (law-abiding) transacted with Bob. Authorities investigate Bob. Bob claims he never received Alice's identity and cannot produce it – hoping to deny knowledge of his counterparties, or to shield Alice from being identified as his customer.

2.5.2 Why It Fails

The `IdentityExchange(alice, bob, E_bob)` event is on-chain and immutable. The court retrieves `E_bob` directly from the blockchain – Bob's cooperation is not needed to find the ciphertext.

Bob is compelled to decrypt:

$$M = C_b - sk_{bob} * R_b$$

If Bob claims he "lost" his private key:

- The court observes that Bob's address continues to transact on Ethereum (signing transactions requires the corresponding private key).

- The court holds Bob in contempt, as it would for any refusal to produce subpoenaed records.
- Even without Bob’s cooperation, the court can establish from the on-chain proof verification that `E_bob` *does* contain a genuine identity. Bob’s refusal to decrypt is itself evidence of obstruction.

Bob cannot claim `E_bob` is meaningless or fabricated: the successful Chaum-Pedersen proof verification is an on-chain fact.

2.5.3 What the Court Sees

`E_bob` is on-chain. The proof that it contains Alice’s real identity is on-chain. Bob decrypts or faces contempt. The cryptographic record is self-authenticating.

2.6 Attack 6: Eve Links Alice’s Identities Across Counterparties

2.6.1 Scenario

Eve is a surveillance actor (competitor, stalker, rogue state). She observes that Alice’s address called `approve` for multiple counterparties: Bob, Carol, and Dave. Eve collects the `IdentityExchange` events:

```
IdentityExchange(alice_addr, bob_addr,  E_bob)
IdentityExchange(alice_addr, carol_addr, E_carol)
IdentityExchange(alice_addr, dave_addr,  E_dave)
```

Eve wants to learn Alice’s real identity, or to determine whether these transactions reveal a pattern (e.g., Alice’s spending habits).

2.6.2 Why It Fails (Content Privacy Holds)

Each re-encryption uses independent randomness:

```
E_bob    = (r_1 * G,  r_1 * pk_bob  + I * G)
E_carol  = (r_2 * G,  r_2 * pk_carol + I * G)
E_dave   = (r_3 * G,  r_3 * pk_dave  + I * G)
```

The ciphertexts are IND-CPA secure (semantic security of ElGamal): each is computationally indistinguishable from a random pair of curve points. Eve cannot decrypt any of them without the counterparty’s private key. She cannot determine from the ciphertexts alone what identity they encode, nor can she compare them to determine if they encode the same identity.

What Eve does know: all three events originate from `alice_addr`. But this is exactly the information already visible in standard ERC-20 `Approval` events. The encrypted identity blobs add **zero additional linkability** beyond Ethereum’s inherent pseudonymous address model.

What Eve does not know: Alice’s name, jurisdiction, ID number, or any attribute of her identity. She sees an address; she does not see a person.

2.6.3 What Eve Sees

Pseudonymous Ethereum addresses and opaque ciphertexts that are computationally indistinguishable from random. The same view she has of any ERC-20 token, plus meaningless (to her) encrypted blobs.

2.7 Attack 7: Alice Uses a Revoked or Expired Credential

2.7.1 Scenario

Alice’s identity credential was revoked by the issuer (e.g., her Alberta residency expired, fraud was detected in her original KYC documents, or she is subject to new sanctions). Alice still holds her old `E_alice` and `sigma` and attempts to continue transacting.

2.7.2 Why It Fails

The `IdentityRegistry` tracks credential validity. When the issuer revokes Alice’s credential:

```
identityRegistry.revokeIdentity(alice_address)
```

Subsequent calls to `isVerified(alice)` return false. Alice’s `approve` call – and any `transfer` or `transferFrom` – reverts at the identity check, before the Chaum-Pedersen proof is even evaluated.

Alice’s old credential and proofs remain mathematically valid (the cryptography is eternal), but the contract refuses to accept them because her registration is no longer active. The revocation is a separate on-chain state, controlled by the trusted issuer.

2.7.3 What the Contract Sees

`isVerified(alice) == false`. Transaction reverts immediately. The still-valid-but-revoked credential is never consulted.

2.8 Attack 8: Unwitting Bob Receives a Transaction From Criminal Alice

2.8.1 Scenario

Bob is a law-abiding merchant running a BUCK-accepting storefront. Alice is a criminal who passed KYC (perhaps before she was sanctioned, or under a jurisdiction that has not yet flagged her). Alice sends BUCK to Bob through a normal transaction. Bob has no way to know Alice is a criminal at the time of the transaction.

Later, authorities investigate Alice. Bob is concerned: did he unknowingly participate in money laundering? Can Alice’s identity in the receipt exonerate him?

2.8.2 Why the Scheme Protects Bob

Bob received Alice's re-encrypted identity `E_bob` during the `approve` handshake (or Bob pre-approved Alice via `approve(alice, 0, ...)` to receive from her). The Chaum-Pedersen proof verified on-chain, so `E_bob` provably contains Alice's real, issuer-certified identity.

When authorities investigate:

1. Bob decrypts `E_bob` and produces Alice's identity `I`.
2. The on-chain proof record establishes that `I` is genuine (not something Bob fabricated to frame Alice, and not something Alice fabricated to deceive Bob).
3. Bob can demonstrate he had no way to know Alice was a criminal: the identity he received was a valid, issuer-signed credential at the time of the transaction. The revocation (if any) came later.
4. Bob's good faith is supported by the cryptographic record: he accepted a transaction from a verified participant and holds a court-presentable receipt.

The system protects honest participants by creating a tamper-proof record of who they transacted with. Bob is not liable for Alice's criminality; he is protected by the same cryptographic chain that convicts Alice.

2.8.3 What the Court Sees

Bob holds a provably genuine receipt. Alice's identity was valid at transaction time. Bob acted in good faith. The receipt is evidence of Bob's compliance, not his complicity.

2.9 Summary: Why Chaum-Pedersen Defeats All Substitution Attacks

Every attack that involves substituting false identity data fails for the same fundamental reason: the Chaum-Pedersen proof is **sound**. A valid proof for a false statement does not exist.

The proof asserts:	<code>E_bob</code> encrypts the same <code>M</code> as <code>E_alice</code>
The contract reads:	<code>E_alice</code> from the registry (issuer-signed, immutable)
Therefore:	<code>E_bob</code> must encrypt Alice's real identity
Alice cannot:	supply a different <code>E_alice</code> (contract reads from storage)
Alice cannot:	produce a valid proof for different <code>M</code> (soundness)
Alice cannot:	forge the issuer's signature (ECDSA unforgeability)
Bob cannot:	produce a different plaintext (ElGamal decryption is deterministic)
Eve cannot:	decrypt <code>E_bob</code> (she lacks <code>sk_bob</code>)

Two independent hardness assumptions protect the system:

1. **Signature unforgeability** (ECDSA): prevents forged credentials

2. **Discrete logarithm hardness** (alt_bn128): prevents false proofs and unauthorized decryption

Both assumptions also underpin Ethereum itself. If either breaks, Ethereum’s own security fails first – the Alberta Buck identity system does not introduce any novel cryptographic assumptions.

3 Identity Visibility Modes

Mode	What on-chain reveals	What off-chain reveals
Anonymous-valid	Address is verified (bit) isVerified(addr) == true	Encrypted receipt: identities of both parties, decryptable only by participants + optional regulatory escrow key
Public	Address is verified + public identity claim on ONCHAINID	Full name, jurisdiction, credentials, track record – world-readable
Contract	Contract address is verified; linked to deployer’s identity	Deployer’s identity is the contract’s identity; receipts encrypted to deployer
Permissionless	No identity check	N/A

4 Operations Table

4.1 Core BUCK (ERC-20) Operations

Operation	Caller identity	Counterparty identity	Receipt	Notes
mint	Anonymous-valid KYC (topic 1)	N/A (self-mint)	Event log	Caller must hold BuckCredits; identity verified but not disclosed on-chain
burn	Anonymous-valid KYC (topic 1)	N/A (self-burn)	Event log	Reduces outstanding balance
transfer	Anonymous-valid KYC (topic 1)	Anonymous-valid KYC (topic 1)	Encrypted 3-party	Both parties verified; receipt encrypted to sender + receiver + escrow
transferFrom	Anonymous-valid KYC (topic 1)	Anonymous-valid KYC (topic 1)	Encrypted 3-party	Spender verified; owner and recipient identities in encrypted receipt
approve	Anonymous-valid KYC (topic 1)	Anonymous-valid KYC (topic 1)	Event log	Approving a spender for transferFrom

4.2 BuckCredit (ERC-721) Operations

Operation	Caller identity	Counterparty identity	Receipt	Notes
createCredit	Public KYC + insurer (42)	Anonymous-valid KYC (topic 1)	Encrypted to parties	Insurer MUST be publicly id (their reputation backs the c Client may be anonymous-v
activate	Anonymous-valid KYC (topic 1)	N/A (self)	Event log	Credit owner activates porti
updateCredit	Public KYC + insurer (42)	N/A	Event log	Insurer updates params; mu (reappraisal is a public com
transfer (NFT)	Anonymous-valid KYC (topic 1)	Anonymous-valid KYC (topic 1)	Encrypted 3-party	Transferring insurance NFT inherits the credit

4.3 BuckKController (PID Stabilization)

Operation	Caller identity	Receipt	Notes
compute	Permissionless	Event log	Anyone can trigger PID update; minter pays gas. No identity needed.
setGains	Governance	Event log	Governance multisig or vote result. Governance members publicly identified.
addBasketComponent	Governance	Event log	Adding a new commodity to the basket.
setDT	Governance	Event log	Changing the PID update interval.

4.4 Oracle Operations

Operation	Caller identity	Receipt	Notes
Submit claim	Public KYC + reporter (43)	Public log	"I, Perry Kundert, claim Gold was \$2905 at block 19400000." Public identity is the entire point: reputation accrues.
Commit value (L1)	Public KYC + reporter (43)	Event log	Authorized reporters sign the commitment. Signatures are publicly verifiable.
Challenge/dispute	Anonymous-valid KYC (topic 1)	Evidence on-chain	Challenger need not be public; the evidence speaks for itself.
Governance vote (oracle selection)	Anonymous-valid KYC (topic 1)	Vote record	Stake-weighted vote on oracle selection. Vote weight from Buck holdings is public; voter identity may be private.

4.5 DeFi Integration (Uniswap, Lending, etc.)

Operation	Caller identity	Contract identity	Receipt	Notes
Create pool (BUCK/USDT)	Public KYC (topic 1)	Contract registered to deployer's identity	Event log	Pool deployer is publicly Contract address linked to ONCHAINID in IdentityRegistry
Swap BUCK -> USDT	Anonymous-valid KYC (topic 1)	Contract (deployer)	Encrypted 3-party	User sends BUCK to pool Receipt: user + deployer
Swap USDT -> BUCK	Anonymous-valid KYC (topic 1)	Contract (deployer)	Encrypted 3-party	Pool sends BUCK to user
Add liquidity	Anonymous-valid KYC (topic 1)	Contract (deployer)	Encrypted 3-party	User transfers BUCK to pool
Remove liquidity	Anonymous-valid KYC (topic 1)	Contract (deployer)	Encrypted 3-party	Pool transfers BUCK to user

4.6 Identity Management

Operation	Caller identity	Issuer identity	Receipt	Notes
KYC ceremony	Self (no claims yet)	Trusted issuer (publicly identified)	Off-chain encrypted	Participant submits docs to issuer issuer verifies, signs claim.
Register in registry	Registry agent	N/A	Event log	Links wallet address to ONCHAINID
Add public identity claim	Self KYC (topic 1)	Trusted issuer (publicly identified)	On-chain claim	Opts in to public identification for oracle/governance participation.
Revoke claim	Issuer or self	N/A	Event log	Removes a claim (e.g., expired license)
N/M recovery	N-of-M parties (all identified)	N/A	On-chain + off-chain	Transfers identity to new address N of M designated parties agree

5 Contract Identity: How DeFi Pools Become BUCK Counterparties

A Uniswap pool contract is just an address. For BUCK transfers to/from that address to pass `isVerified`, the contract must have an identity in the IdentityRegistry. But a contract can't do KYC. The solution:

The pool deployer's identity becomes the contract's identity.

When someone creates a BUCK/USDT Uniswap pool:

1. Deployer (a verified BUCK participant) calls the Uniswap factory to create the pool.
The factory deploys a new contract at a deterministic address.
2. Deployer (or a registry agent) calls: `identityRegistry.registerIdentity(poolAddress, deployer.onchainId, country)`
This links the pool contract's address to the deployer's ONCHAINID. The pool inherits the deployer's identity claims.
3. Now `isVerified(poolAddress)` returns true. BUCK transfers to/from the pool pass the identity check.

4. Encrypted receipts for pool transactions list the deployer’s identity as the counter-party, because the deployer is the legal operator of the pool.

This means: if you deploy a BUCK DeFi pool, you are the identified legal operator. Your identity is on the line for every transaction through that pool. If a court subpoenas you, you can be compelled to decrypt the receipts (because your key is one of the encryption parties).

6 The Receipt Problem: What Can Be Public, What Cannot

Events are public. Every `emit` on Ethereum is readable by every node. The previous design emitted `ONCHAINID` addresses and ECDH public keys in receipt events – this would create a permanent, public, linkable identity graph of every BUCK transfer. That defeats the entire privacy model.

The core constraints:

1. **The EVM is transparent:** every contract execution, every storage slot, every emitted event is publicly reconstructible. The BUCK contract cannot hold decrypted identity data, even transiently.
2. **The contract is trusted:** the BUCK contract enforces `isVerified` and can verify proofs. What it cannot do is learn secrets.
3. **The identity must be re-encrypted:** when Alice wants to share her identity with Bob (for a receipt), she must produce a ciphertext that Bob can decrypt but nobody else can – and the BUCK contract must be confident this ciphertext contains Alice’s **real** identity, not garbage.
4. **DeFi timing:** in a `transferFrom` call, the router cannot pass receipt data. Identity exchange must happen earlier (at `approve` time).

6.1 The Fundamental Challenge: Proving Re-Encryption Correctness

Alice’s identity `I` is stored encrypted on Holochain, committed on-chain as `C = hash(I, salt)` (a Pedersen commitment or similar). When she wants to share her identity with Bob for a transfer receipt, she must:

1. Decrypt `I` (she has her private key)
2. Re-encrypt `I` under Bob’s public key: `E_bob = Encrypt(I, ECDH(k_alice, K_bob))`
3. Prove to the BUCK contract that `E_bob` encrypts the **same** `I` that `C` commits to – **without revealing `I` to the contract**

This is a **zero-knowledge proof of correct re-encryption**.

6.1.1 What the ZK Proof States

Public inputs (visible to contract and everyone):

```
C          -- Alice's identity commitment (on-chain, from ONCHAINID)
E_bob      -- Alice's identity re-encrypted for Bob
K_bob      -- Bob's public key (from his ONCHAINID or from approve)
```

Private inputs (known only to Alice):

```
I          -- Alice's plaintext identity
salt       -- commitment randomness
k_alice    -- Alice's private key
```

Proof statement:

```
C == Commit(I, salt)
AND E_bob == Encrypt(I, ECDH(k_alice, K_bob))
```

The contract verifies the ZK proof (200K-300K gas for Groth16 SNARK verification). If valid, the contract knows $\sim E_bob$ is a correct re-encryption of the committed identity. The contract never learns I .

6.1.2 Proxy Re-Encryption (PRE): An Alternative

NuCypher's Umbral scheme¹ provides threshold proxy re-encryption where a proxy transforms a ciphertext from Alice's key to Bob's key without decrypting. Umbral includes non-interactive zero-knowledge (NIZK) proofs of re-encryption correctness.

In the Alberta Buck context:

1. Alice's identity is encrypted under her key: $E_alice = \text{Encrypt}(I, K_alice)$
2. Alice computes a re-encryption key: $rk_{\{A \rightarrow B\}} = \text{ReKeyGen}(k_alice, K_bob)$
3. The Holochain hApp (acting as proxy) applies:
 $E_bob = \text{ReEncrypt}(rk_{\{A \rightarrow B\}}, E_alice)$
4. The hApp produces a NIZK proof that re-encryption was correct
5. Alice submits (E_bob, proof) to the BUCK contract
6. Contract verifies the proof against E_alice (committed on-chain)

The proxy (Holochain hApp) never sees I . It transforms one ciphertext into another without decrypting. The NIZK proof lets the contract verify correctness.

6.1.3 Why Holochain for Proof Generation

ZK proof generation is computationally expensive (seconds to minutes for complex circuits). Holochain provides:

¹Umbral: A Threshold Proxy Re-Encryption Scheme. NuCypher. Reference implementation: <https://github.com/nucypher/pyUmbral> Whitepaper: <https://raw.githubusercontent.com/nucypher/umbral-doc/master/umbral-doc.pdf>

- **Trusted code:** the hApp DNA is identified by hash; anyone can verify the exact code that generated the proof. Unlike a random off-chain service, the hApp code is deterministic and auditable.
- **No gas cost:** proof generation runs on the participant's Holochain node, not on Ethereum. The only on-chain cost is proof verification.
- **Agent-centric storage:** Alice's identity data lives on her own source chain, encrypted. The hApp decrypts locally, re-encrypts for Bob, generates the proof – all within Alice's node. Nothing leaves her node except `E_bob` and the proof.

However, Holochain is NOT sufficient alone for trust. Alice runs her own node and could theoretically run modified hApp code. The ZK proof is the trust anchor: it is mathematically verifiable on-chain regardless of what software generated it. Holochain provides the execution environment; the ZK proof provides the guarantee.

6.2 Homomorphic Equality Check Without Decryption

Perry's question: "Is there a homomorphic encryption process by which we can compare an encrypted value as equal, but the party checking for equality cannot decrypt the data?"

Yes. Several approaches:

6.2.1 Pedersen Commitments

$\text{Commit}(v, r) = v \cdot G + r \cdot H$ where G, H are generator points. The commitment hides v (information-theoretically) but binds to it (computationally, under discrete log assumption).

To prove two commitments $C_1 = \text{Commit}(v, r_1)$ and $C_2 = \text{Commit}(v, r_2)$ contain the same value:

Proof: reveal $\text{delta}_r = r_1 - r_2$

Verifier checks: $C_1 - C_2 == \text{delta}_r * H$

The verifier learns nothing about v , only that both commitments bind to the same value. This is efficient (~ 1 EC point addition).

6.2.2 Pedersen + ZK Range Proof (Bulletproofs)

If the identity is a structured value (not just a scalar), Bulletproofs² can prove properties of committed values without revealing them: range proofs, equality proofs, membership proofs. Verification is $O(\log n)$ and practical on-chain.

²Bulletproofs: Short Proofs for Confidential Transactions and More. Bunz, Bootle, Boneh, Poelstra, Wuille, Maxwell. Overview: <https://tlu.tarilabs.com/cryptography/the-bulletproof-protocols> Applied to Pedersen commitments: <https://www.nccgroup.com/research/on-the-use-of-pedersen-commitments-for-confidential-payments/>

6.2.3 Application to Re-Encryption Verification

The BUCK contract stores Alice's identity commitment: `C_alice = Commit(I, r_alice)`

Alice provides a re-encrypted blob `E_bob` and a ZK proof that the plaintext of `E_bob` is the same `I` committed in `C_alice`.

The contract verifies the proof. It never learns `I`. It learns only that `E_bob` contains the correct identity, re-encrypted for Bob.

6.3 The approve Function as Identity Handshake

The ERC-20 `approve` function is the natural point for identity exchange. Alice is directly calling the BUCK contract – she can provide the re-encrypted identity blob and the ZK proof:

```
function approve(
    address spender,
    uint256 amount,
    bytes    calldata reEncryptedIdentity,    // E_bob: her identity for spender
    bytes    calldata reEncryptionProof      // ZK proof of correct re-encryption
) public returns (bool) {
    _approve(msg.sender, spender, amount);

    // Verify the re-encryption proof against the caller's on-chain commitment
    bytes32 identityCommitment = identityRegistry.identityCommitment(msg.sender);
    require(
        _verifyReEncryption(identityCommitment, reEncryptedIdentity, reEncryptionProof),
        "Invalid re-encryption proof"
    );

    // Store the verified re-encrypted identity for this pair
    _receiptFragments[msg.sender][spender] = keccak256(reEncryptedIdentity);

    // Emit ONLY the re-encrypted blob (opaque to everyone except spender)
    emit IdentityExchange(msg.sender, spender, reEncryptedIdentity);
    return true;
}
```

What the public sees in the `IdentityExchange` event:

- Alice's address approved a spender
- An opaque blob (`reEncryptedIdentity`) that only the spender can decrypt
- **No** ONCHAINID address, **no** public key, **no** linkable identifier

What the contract verified:

- The blob contains Alice’s real identity (ZK proof passed)
- The identity matches her on-chain commitment

What the spender (Bob / pool operator) can do:

- Decrypt the blob using `ECDH(k_bob, K_alice)` to learn Alice’s identity
- Store it for receipt construction

6.3.1 Receiving Transfers: `approve(sender, 0)` Handshake

For an account that wants to **receive** from a specific counterparty:

```
// Alice wants to receive BUCK from Bob (Alice is private-identity):
buck.approve(bob, 0, reEncryptedIdentityForBob, proof)
```

Amount = 0 means no spending authorization. The side-effect: Alice’s re-encrypted identity is now stored and verified for Bob. When Bob later calls `transfer(alice, amount)`, the BUCK contract knows Alice’s identity fragment exists and is verified.

6.4 Receipt Construction: What the Contract Emits

With the corrected design, the BUCK contract emits a **minimal** event that reveals only pseudonymous addresses (which the ERC-20 Transfer event already reveals) plus a reference to the pre-stored identity fragments:

```
event BuckTransferReceipt(
    address indexed from,
    address indexed to,
    uint256 amount,
    bytes32 fromIdentityHash,    // hash of from’s re-encrypted identity
    bytes32 toIdentityHash      // hash of to’s re-encrypted identity
);
```

The `fromIdentityHash` and `toIdentityHash` are hashes of the opaque re-encrypted blobs stored during `approve`. They add **no linkability** beyond what the Transfer event already provides (the addresses themselves are the same).

The actual re-encrypted identity blobs are retrievable:

- From the `IdentityExchange` events (emitted during `approve`)
- From Holochain (stored on the participants’ source chains)

Only the counterparty (who has the private key to decrypt the blob) can reconstruct the full receipt with real identities.

6.5 The Complete Flow

```
function transfer(address to, uint256 amount) public override returns (bool) {
    require(identityRegistry.isVerified(msg.sender), "Sender not verified");
    require(identityRegistry.isVerified(to), "Receiver not verified");
    require(compliance.canTransfer(msg.sender, to, amount), "Compliance");

    _transfer(msg.sender, to, amount);
    compliance.transferred(msg.sender, to, amount);

    // Emit minimal receipt -- no identity data, just hashes of
    // pre-stored re-encrypted blobs
    emit BuckTransferReceipt(
        msg.sender, to, amount,
        _receiptFragments[msg.sender][to],
        _receiptFragments[to][msg.sender]
    );

    return true;
}
```

The `transferFrom` path is identical: the receipt fragment hashes were stored during `approve`, so no additional data passes through the DeFi router.

6.6 Summary: What Each Layer Sees

Layer	What it sees
Ethereum (everyone)	Transfer(from, to, amount) – pseudonymous addresses BuckTransferReceipt – same addresses + opaque hashes IdentityExchange – address + opaque encrypted blob ZK proof verified (pass/fail) – no identity content
BUCK contract	isVerified(from) = true , isVerified(to) = true ZK proof of re-encryption correctness (verified) Hash of re-encrypted identity blob (stored) Never sees plaintext identity
Counterparty (Bob)	Decrypts re-encrypted blob -> learns Alice's identity Constructs full receipt (names, details)
Sender (Alice)	Knows her own identity, knows Bob's (from his approve) Constructs full receipt
Holochain hApp	Generates re-encrypted blob + ZK proof locally Stores encrypted receipts on source chains Code identified by DNA hash (auditable)
Court (with subpoena)	Compels Bob to decrypt his copy of Alice's identity Bob reveals receipt without revealing private key

7 Cryptographic Receipts: 3-Party ECDH

The BUCK contract emits trusted plaintext receipt events (previous section). The wallet layer encrypts these into 3-party ECDH receipts that only the participants can decrypt.

7.1 The Three Keys

For a transfer from Alice to a DeFi pool operated by Bob:

Key	Held by	Purpose
<code>k_s</code> (sender)	Alice	Sender's persistent signing/encryption key
<code>k_e</code> (ephemeral)	Deterministic	Derived from hash of trusted receipt event; reproducible by anyone who knows the event
<code>K_r</code> (receiver)	Bob (deployer)	Pool operator's persistent public key

The ephemeral key `k_e` is derived deterministically from the trusted receipt event (which is on-chain and immutable):

```
// The receipt event fields are all on-chain, all trusted:
receipt_data = abi.encode(
    from, to, amount,
    fromIdentity, toIdentity,
    fromPubKey, toPubKey,
    block.number
)
k_e = keccak256(receipt_data)    // deterministic "private key"
```

7.2 Encryption Scheme

The wallet computes the encryption key via ECDH:

```
// Alice's wallet computes (knows k_s, k_e, K_r):
shared_1 = ECDH(k_s, K_r)        // sender-receiver shared secret
shared_2 = ECDH(k_e, K_r)        // ephemeral-receiver shared secret
encryption_key = keccak256(shared_1 || shared_2)

// Augment the trusted receipt with off-chain identity data:
full_receipt = {
    ...receipt_data,              // from the on-chain event (trusted)
    sender_name: "Alice Johnson", // from ONCHAINID private claim
    receiver_name: "Bob Smith",   // from ONCHAINID claim (via Holochain)
    purpose: "Uniswap V3 BUCK->USDT swap"
}

// Encrypt:
ciphertext = AES-GCM(encryption_key, encode(full_receipt))
```

The encrypted receipt is stored on Holochain (both parties' source chains). A commitment hash can optionally be emitted on-chain:

```
event ReceiptCommitment(  
    bytes32 indexed transferHash,    // hash of the BuckTransferReceipt event  
    bytes32 receiptHash             // hash of the encrypted receipt  
);
```

7.3 Who Can Decrypt

7.3.1 Sender (Alice)

Alice has k_s and can recompute k_e from the on-chain receipt event:

```
k_e = keccak256(receipt_event_data)    // deterministic from on-chain data  
shared_1 = ECDH(k_s, K_r)              // she has k_s, K_r from event  
shared_2 = ECDH(k_e, K_r)              // she has k_e, K_r from event  
encryption_key = keccak256(shared_1 || shared_2)
```

7.3.2 Receiver / Pool Operator (Bob)

Bob has k_r (private key) and reads the sender's public key and the ephemeral public key from the on-chain event:

```
K_e = G * k_e                          // computable from event data  
shared_1 = ECDH(k_r, K_s)              // K_s = fromPubKey from event  
shared_2 = ECDH(k_r, K_e)              // K_e from event data  
encryption_key = keccak256(shared_1 || shared_2)
```

7.3.3 Court-Compelled Decryption (Subpoena of Pool Operator)

A court subpoenas Bob to decrypt a specific transaction receipt:

```
// Court provides: the transaction hash  
// Bob looks up: the BuckTransferReceipt event from that transaction  
// Bob computes (using his private key k_r):  
shared_1 = ECDH(k_r, K_s)              // K_s from event  
shared_2 = ECDH(k_r, K_e)              // K_e from event  
encryption_key = keccak256(shared_1 || shared_2)  
receipt = AES-GCM-decrypt(encryption_key, ciphertext_from_holochain)
```

Bob reveals only the decrypted receipt. He never reveals k_r . He cannot decrypt receipts where he is not a party.

7.3.4 Regulatory Escrow (Optional)

A regulatory authority's public key `K_reg` adds a third shared secret:

```
shared_3 = ECDH(k_e, K_reg)
encryption_key = keccak256(shared_1 || shared_2 || shared_3)
```

The regulator can compute `shared_3` (they have `k_reg`, and `K_e` is deterministic from the on-chain event). But they still need `shared_1` or `shared_2` – requiring cooperation from a transaction participant. No unilateral surveillance.

8 Play-by-Play: BUCK/USDT Swap on Uniswap

Alice holds 1000 BUCK and wants to swap 500 BUCK for USDT. Bob has deployed a BUCK/USDT Uniswap V3 pool.

8.1 Prerequisites (One-Time Setup)

Alice:

1. Completed KYC with trusted issuer (off-chain)
2. Trusted issuer signed claim (topic 1: KYC) on Alice's ONCHAINID
3. Registry agent called `identityRegistry.registerIdentity(alice_wallet, alice_onchainid, country_code)`
4. `identityRegistry.isVerified(alice_wallet) == true`

Bob (pool deployer):

1. Completed KYC (off-chain)
2. Has KYC claim on his ONCHAINID
3. Is registered in IdentityRegistry
4. Created BUCK/USDT pool via Uniswap factory
-> pool deployed at address `0xPOOL`
5. Registered pool in IdentityRegistry:
`identityRegistry.registerIdentity(0xPOOL, bob_onchainid, country_code)`
6. `identityRegistry.isVerified(0xPOOL) == true`

8.2 The Swap Transaction

8.2.1 Step 1: Alice Approves the Router (Identity Handshake)

Alice's wallet invokes the Holochain `hApp` to prepare the identity exchange, then calls `approve` on the BUCK contract:

```
// 1. Off-chain (Holochain hApp on Alice's node):
//   - Decrypt Alice's identity I from her source chain
//   - Look up the pool operator's (Bob's) public key K_bob
```

```
// - Re-encrypt: E_bob = Encrypt(I, ECDH(k_alice, K_bob))
// - Generate ZK proof: "E_bob encrypts the same I committed on-chain"

// 2. On-chain:
buck.approve(
    uniswapRouter,      // spender (the router, which calls transferFrom)
    500e18,              // amount
    E_bob,              // Alice's identity re-encrypted for Bob
    proof                // ZK proof of correct re-encryption
)
```

The BUCK contract:

1. Sets allowance[alice][uniswapRouter] = 500e18
2. Reads Alice's identity commitment from IdentityRegistry
3. Verifies the ZK proof: E_bob correctly re-encrypts the committed I
4. Stores hash(E_bob) in _receiptFragments[alice][0xPOOL]
5. Emits IdentityExchange(alice, 0xPOOL, E_bob)
 - the event contains ONLY the opaque encrypted blob
 - no ONCHAINID address, no public key, no linkable identifier

This is the critical moment. Alice is directly calling the BUCK contract, so she can provide the re-encrypted identity and ZK proof. By the time `transferFrom` is called later (by the router), the verified identity fragment is already stored.

8.2.2 Step 2: Alice Calls the Router

```
// Alice initiates the swap (standard Uniswap call -- no BUCK-specific data)
uniswapRouter.exactInputSingle(
    tokenIn:  BUCK,
    tokenOut: USDT,
    fee:       3000,          // 0.3% fee tier
    recipient: alice,
    amountIn:  500e18,
    amountOutMinimum: 490e6, // slippage protection
    sqrtPriceLimitX96: 0
)
```

No identity data passes through this call. The router is a standard Uniswap contract that knows nothing about BUCK identity.

8.2.3 Step 3: Router Calls BUCK.transferFrom (Minimal Receipt Emission)

Inside the router, the swap triggers:

```
buck.transferFrom(alice, 0xPOOL, 500e18)
```

The BUCK contract executes (no identity data passes through the router):

```
function transferFrom(address from, address to, uint256 amount) ...
{
    _spendAllowance(from, msg.sender, amount);
    require(identityRegistry.isVerified(from), "Sender not verified");
    require(identityRegistry.isVerified(to), "Receiver not verified");
    require(compliance.canTransfer(from, to, amount), "Compliance");

    _transfer(from, to, amount);
    compliance.transferred(from, to, amount);

    // Emit minimal receipt -- ONLY hashes of pre-stored opaque blobs
    emit BuckTransferReceipt(
        from, to, amount,
        _receiptFragments[from][to], // hash of Alice's E_bob
        _receiptFragments[to][from] // hash of Bob's E_alice (if exists)
    );

    return true;
}
```

What the public sees: the same from/to/amount as the standard ERC-20 Transfer event, plus two opaque hashes. No ONCHAINID addresses. No public keys. No linkable identity information beyond the pseudonymous Ethereum addresses.

What the contract verified (during approve, earlier): the ZK proof that Alice's re-encrypted identity blob is a correct re-encryption of her committed identity.

8.2.4 Step 4: Wallet Constructs the Full Receipt (Off-Chain)

Alice's wallet (Holochain hApp) observes the BuckTransferReceipt event and the IdentityExchange events to construct the full receipt:

```
// 1. From on-chain events (trusted, immutable):
transfer_data = {from, to, amount, block.number, tx_hash}

// 2. From IdentityExchange event (emitted during approve):
E_bob = Alice's re-encrypted identity for Bob (opaque blob)

// 3. From Holochain (Alice's source chain):
alice_identity = decrypt(alice_encrypted_identity, k_alice)

// 4. From Bob's IdentityExchange (if Bob also did approve for Alice):
E_alice = Bob's re-encrypted identity for Alice
bob_identity = decrypt(E_alice, k_alice) // Alice can decrypt this
```

```
// 5. Construct full receipt
full_receipt = {
    transfer:      transfer_data,
    sender_name:   alice_identity.name,      // "Alice Johnson"
    receiver_name: bob_identity.name,        // "Bob Smith, DeFi Ops Inc."
    operation:     "Uniswap V3 swap BUCK -> USDT",
    pool:         "0xPOOL",
    timestamp:     block.timestamp
}

// 6. Encrypt the full receipt with 3-party ECDH (see next section)
k_e = keccak256(abi.encode(transfer_data))
shared_1 = ECDH(k_alice, K_bob)
shared_2 = ECDH(k_e, K_bob)
encryption_key = keccak256(shared_1 || shared_2)
ciphertext = AES-GCM(encryption_key, encode(full_receipt))

// 7. Store on Holochain (both parties' source chains)
holochain.store(alice_chain, ciphertext)
holochain.store(bob_chain,  ciphertext)
```

8.2.5 Step 5: Pool Executes the Swap

The Uniswap pool sends USDT to Alice. Since USDT is not an Alberta Buck token, the USDT transfer has no BUCK identity requirements (it follows whatever rules USDT imposes, which is typically none).

8.2.6 Step 6: Completed State

On-chain (public):

- ERC-20 Transfer event: `alice_addr -> 0xPOOL, 500 BUCK`
- `BuckTransferReceipt`: same addresses + two opaque bytes32 hashes
- `IdentityExchange` (from approve): `alice_addr + opaque encrypted blob`
- Swap event: `BUCK/USDT pool, 500 BUCK in, ~495 USDT out`
- ZK proof verification result: pass (no identity content leaked)

Off-chain (Holochain, encrypted):

- Full encrypted receipt on Alice's and Bob's source chains
- Alice's re-encrypted identity blob (decryptable only by Bob)
- Bob's re-encrypted identity blob (decryptable only by Alice)

What anyone can see:

- Two pseudonymous addresses transacted 500 BUCK
- Both addresses pass `isVerified` (they are KYC'd participants)

- Opaque blobs and hashes that reveal nothing without private keys
- A ZK proof was verified (the identity blobs are genuine)

What Alice can see:

- Her own identity (trivially)
- Bob's identity (decrypt his re-encrypted blob with her key)
- Full receipt with names, amounts, purpose

What Bob can see:

- His own identity (trivially)
- Alice's identity (decrypt her re-encrypted blob with his key)
- Full receipt with names, amounts, purpose

What a court can compel (via subpoena of Bob):

- Bob decrypts Alice's blob for the specific transaction
- Reveals Alice's identity for that one receipt
- Bob never reveals his private key

What the public CANNOT see:

- Alice's real name or ONCHAINID
- Bob's real name (as pool operator)
- Any linkable identity across transactions
- The purpose or terms of the transaction

9 Edge Cases and Design Considerations

9.1 Flash Loans and Atomic Transactions

A flash loan borrows and repays in a single transaction. The borrower's address must be verified, and the lending contract must be registered. The receipt covers the full borrow-repay cycle as a single atomic event.

9.2 Multi-Hop Swaps

A swap routed through BUCK -> WETH -> USDT involves two BUCK transfers (BUCK -> pool1, pool1 -> pool2 if intermediate). Each BUCK transfer generates its own encrypted receipt. The router contract must also be registered if it holds BUCK transiently.

9.3 Contract-to-Contract Transfers

When one DeFi contract sends BUCK to another (e.g., a yield aggregator moving BUCK between pools), both contracts must be registered. The receipt identifies both contracts' deployers as the parties. This creates a clear chain of custody: every BUCK movement has an identified legal operator at each end.

9.4 Wrapping and Bridging

If BUCK is wrapped (WBUCK) for use on another chain, the wrapping contract must be registered. The wrap/unwrap generates receipts. On the destination chain, the bridged BUCK may operate under different identity rules (or no identity rules if the destination chain doesn't enforce ERC-3643). This is a known limitation – the Alberta Buck's identity guarantees only hold on chains where the IdentityRegistry is deployed.

9.5 Gas Costs

The corrected design has two gas-consuming operations:

Operation	Cost ($\sim$ gas)	When
approve with ZK proof verify	$\sim$ 200K-300K gas	Once per pair
BuckTransferReceipt event (2 indexed addresses + 3 uint256)	$\sim$ 2,000 gas	Per transfer

The ZK proof verification (**approve**) is expensive but happens once per counterparty pair. Subsequent transfers between the same pair emit only the minimal receipt event (addresses + hashes), which is cheap.

The encrypted receipt itself is stored on Holochain, not on-chain. No encrypted blobs are emitted as Ethereum events.

9.6 Privacy of Pool Deployer Identity

The pool deployer's identity is linked to the pool contract in the IdentityRegistry, but this mapping is on-chain and public. Anyone can look up `identityRegistry.identity(0xPOOL)` and find Bob's ONCHAINID.

If Bob's ONCHAINID has only private claims (KYC but not public identity), then the public knows "0xPOOL is operated by a verified entity" but not WHO that entity is. Bob's identity is revealed only in encrypted receipts.

If Bob has a public identity claim (e.g., "DeFi Ops Inc., Alberta"), then anyone can see who operates the pool. This is likely the expected case for commercial DeFi operators – public identity builds trust and attracts liquidity.

10 Summary: Identity Requirement by Risk Level

Risk level	Operations	Identity mode	Receipt
Highest	BuckCredit creation/update, oracle claims, governance	Public	On-chain event + public record
High	Pool deployment, compliance changes	Public	On-chain event
Medium	BUCK transfer, DeFi swap, add/remove liquidity	Anonymous-valid	Encrypted 3-party ECDH receipt
Low	BUCK burn, credit activation	Anonymous-valid	On-chain event
None	BuckK compute, read oracle, view balances	Permissionless	On-chain event